PRISM

Test Approach Documentation

Project	PRISM Monitor
Authors	Elizabeth Herrera Roman Huerta Reece Schenck
Date	2026-04-10

1. Introduction

This document describes the testing approach for the PRISM Monitor project. PRISM is a full-stack system monitoring platform composed of a Python metric collector agent, a Node.js API server, a MySQL database, and a React dashboard. The goal of testing is to verify that the product collects accurate system metrics, stores and retrieves data correctly, displays information properly in the dashboard, and can be packaged and deployed successfully as a Debian package.

The approach in this document covers functional validation, component-level testing, system integration, build and installation verification, performance checks, and failure handling. Because PRISM depends on multiple connected layers working together, the test strategy places strong emphasis on end-to-end validation.

2. Test Objectives

- Validate that CPU, memory, disk, and process metrics are collected accurately from the Linux host.
- Verify that collected data is written to the correct MySQL tables and can be retrieved without corruption.
- Confirm that API endpoints return complete, correct, and timely data to the frontend.
- Ensure that the React dashboard displays latest metrics, historical trends, and process details correctly.
- Verify that authentication, role-based access behavior, and administrative functions operate as intended.
- Confirm that the build script, .deb package, installation scripts, and systemd services work correctly on Ubuntu.

3. Scope of Testing

Testing will cover the following PRISM components and workflows.

Area	Included Components	Primary Focus
Metric Collection	Python agent, Linux commands, scheduler	Accuracy, formatting, logging, error handling
Data Storage	MySQL schema and inserts	Correct table usage, timestamps, record integrity
Backend API	Node.js service and endpoints	Response accuracy, completeness, and stability
Frontend	React dashboard and navigation	Correct rendering, usability, and data presentation

Packaging and Deployment	Build script, .deb package, maintainer scripts	Successful build, install, config, and startup

4. Test Environment

Testing will be performed in an Ubuntu-based environment aligned with the documented build and deployment prerequisites.

Component	Target / Minimum	Purpose
Operating System	Ubuntu 20.04+	Build and deployment platform
Python	3.10+	Metric collection agent
Node.js	18.x	API server and dashboard build
npm	Bundled with Node.js	Dependency installation and frontend build
MySQL	8.0+	Persistent metrics storage
dpkg-dev / apt	Installed	Package creation and installation
systemd	Available	Service enablement and runtime validation

5. Test Types and Approach

5.1 Functional Testing

Function testing will verify that each feature performs its intended task correctly. This is a core part of the PRISM test strategy because the system depends on multiple features behaving correctly on their own before they can work together.

- Validate collection of CPU, memory, disk, and process metrics from the Linux host.
- Verify that collected values are inserted into the correct database tables with valid timestamps.
- Confirm that API endpoints such as metrics/latest, metrics/history, and processes/top return expected data.

- Verify that dashboard gauges, charts, and process views show current and historical data correctly.
- Validate login behavior, rejection of invalid credentials, and proper visibility of admin-only controls.
- Confirm that installation scripts, database initialization, and service startup commands function as expected.

The table below shows representative functional checks that should be included during execution.

ID	Function	How It Runs	Expected Result	Priority	Notes
FT-01	Collect CPU Metrics	Execute Python metric collection script	Valid CPU values and timestamp are recorded	High	Validates end-to-end CPU collection
FT-02	Insert Memory Metrics	Run Python script and inspect MySQL Memory table	Memory record is stored in the correct table	High	Confirms DB write logic
FT-03	Display Dashboard Gauges	Load frontend dashboard in browser	Frontend shows current CPU, memory, and disk values	High	UI correctness
FT-04	Authenticate Valid User	Enter valid credentials in login UI	Authorized user can sign in successfully	High	Security requirement
FT-05	Reject Invalid Login	Enter invalid credentials	Access is denied and an error is shown	High	Prevent unauthorized access
FT-06	Process Table Filtering	Apply filters in UI	Results update correctly	Medium	Sorting + filtering
FT-07	History Graph	Select time range	Graph updates with correct data	Medium	Data visualization accuracy

5.1.1 Unit Testing

Unit testing validates individual functions or modules in isolation to catch logic errors early and reduce debugging effort during integration. By testing the smallest units of code independently, defects can be identified and fixed before they propagate through the system and become more difficult to diagnose.

For the Python collector, unit tests focus on parsing functions that extract metrics from Linux command output such as lscpu, /proc/meminfo, and df. These functions must correctly handle variations in output format and edge cases such as missing values or unexpected data types. Tests use mocked system output to ensure consistent and repeatable results without depending on actual system state.

For the Node.js API, unit tests verify route handlers, response formatting logic, and utility functions such as history range parsing. Tests confirm that functions like parseRange correctly interpret values such as 1h, 24h, and 7d, and that response objects conform to expected JSON structures.

For the React dashboard, unit tests validate individual components such as MetricCard, HistoryChart, and ProcessTable. These tests verify that components render correctly with various input props, handle missing or malformed data gracefully, and respond appropriately to user interactions like sorting and pagination.

The table below shows representative unit checks that should be included during execution.

ID	Function	How It Runs	Expected Result	Priority	Notes
UT-01	Collect CPU Metrics	Run Python script with mocked lscpu output	Valid CPU values and timestamp are recorded	High	Validate parsing logic
UT-02	Memory Metrics Collection	Execute script using sample /proc/meminfo	MemTotal and MemFree correctly extracted	High	Edge cases for missing values
UT-03	Disk Metrics Collection	Run df -h parser	Disk size, used, available correctly mapped	High	Verify % calculations
UT-04	Process Data Extraction	Simulate top command output	Top processes correctly identified and sorted	High	Validate top 20 logic
UT-05	Database Insert Function	Call DB write function with mock data	Data inserted successfully into MySQL tables	High	Verify schema alignment
UT-06	API Endpoint /latest	Call endpoint with mocked DB	Returns most recent record	High	Validate JSON structure

UT-07	API Endpoint /history	Query API with range param	Returns correct time-series data	High	Validate filtering
UT-08	API Endpoint /processes/top	Call endpoint	Returns top processes sorted by CPU	High	Check sorting logic

5.1.2 Integration Testing

Integration testing verifies that the interfaces between PRISM components work correctly when data moves from one layer to another. While unit tests confirm that individual functions operate correctly in isolation, integration tests confirm that these functions work together as expected when connected through real interfaces.

The Python collector to MySQL integration is tested by running the collector against a real database instance and verifying that rows appear in the cpu, memory_info, disk_info, and process_info tables with correct timestamps and values. This confirms that database connection handling, SQL generation, and transaction management work correctly.

The MySQL to Node.js API integration is tested by inserting known test data into the database and calling API endpoints to verify that the returned JSON matches expected values. This confirms that SQL queries, result parsing, and response formatting work together correctly. Special attention is given to the joins required to combine data from separate metric tables.

The API to React frontend integration is tested by running the full stack and verifying that dashboard components display data returned by the API. This confirms that fetch requests, JSON parsing, and state management work correctly across the network boundary.

The authentication flow integration spans all layers: the frontend login form, API authentication endpoint, database user lookup, and session or token management. Tests verify that valid credentials result in successful authentication and that invalid credentials are properly rejected at each layer.

The table below shows representative integration checks that should be included during execution.

ID	Function	How It Runs	Expected Result	Priority	Notes
IT-01	Retrieve Latest Metrics	Call API endpoint /api/metrics/latest	API returns the most recent metric snapshot	High	API and DB interaction
IT-02	Return Process List	Call /api/processes/top and view UI	API and UI show current top processes	High	API and UI consistency

IT-03	Python to MySQL	Run script end-to-end	Data successfully written to DB	High	Validate connection
IT-04	MySQL to API	Insert test data to call API	API returns DB data correctly	High	Data consistency
IT-05	API to React UI	Run full stack	UI reflects API data correctly	High	JSON parsing
IT-06	Authentication Flow	Login to API request	Token used for authorized requests	High	Session validation
IT-07	Error Handling	Stop DB to run API	API returns error gracefully	High	Resilience test

5.1.3 System Testing

System testing validates the complete PRISM workflow as an integrated product in a realistic environment. Unlike integration tests that focus on specific interfaces, system tests exercise the entire application from end to end, confirming that all components work together to deliver expected functionality.

The primary system test verifies the complete data pipeline: the collector samples system metrics on schedule, writes them to MySQL, the API retrieves and formats the data, and the dashboard displays current and historical values. This test confirms that PRISM fulfills its core purpose as a monitoring system.

System tests also verify operational behavior such as service management through systemd. Tests confirm that prism-document-agent and prism-document-web services start correctly, can be stopped and restarted without data loss, and resume normal operation after system reboots.

Installation and deployment testing at the system level verifies that the .deb package installs successfully on a clean Ubuntu system, that all dependencies are satisfied, that configuration files are placed correctly, and that services register and start automatically. This confirms that PRISM can be deployed outside the development environment.

Failure and recovery scenarios are tested at the system level to verify resilience. Tests simulate conditions such as database unavailability, service crashes, and configuration errors to confirm that PRISM fails gracefully with clear error messages and recovers automatically when conditions return to normal.

The table below shows representative system checks that should be included during execution.

ID	Function	How It Runs	Expected Result	Priority	Notes
ST-01	Run package install	Install .deb package using apt	Package installs and services register correctly	High	Deployment validation
ST-02	Full Data Pipeline	Run entire system	Metrics collected, stored, and displayed	Critical	End-to-end validation
ST-03	Performance Load	Simulate high-frequency polling	System remains stable under load	High	Check CPU < 1% goal
ST-04	Data Retention	Insert old data to run cleanup	Old data pruned correctly	Medium	Verify cron cleanup
ST-05	Deployment (.deb install)	Install package via apt	System installs and services start	Critical	Verify dependencies
ST-06	Service Restart	Restart systemd service	System resumes data collection	High	Persistence check
ST-07	Security Testing	Attempt SQL injection	Input sanitized, no DB compromise	Critical	Must pass
ST-08	Logging Verification	Run system	Logs generated for all operations	Medium	Debug support
ST-09	Failure Recovery	Kill Python process	System recovers or logs failure	High	Reliability

5.2 Non-functional testing

Non-functional testing evaluates system attributes that are not related to specific behaviors or functions, but rather to how well the system performs its functions. While functional tests verify what the system does, non-functional tests verify how the system does it. For PRISM, non-functional testing focuses on three critical areas: performance and reliability, security and access control, and build and deployment processes.

These tests are essential because a monitoring system that is functionally correct but consumes excessive resources, exposes security vulnerabilities, or fails to install properly cannot be deployed in production. Non-functional requirements often determine whether a system is viable for real-world use.

5.2.1 Performance and Reliability Testing

Performance and reliability testing verifies that PRISM operates efficiently and remains stable under various conditions. Because PRISM is a monitoring tool, it must have minimal impact on the systems it monitors. The collector agent should consume less than 1% CPU on average, and the dashboard should remain responsive even with large datasets.

These tests also verify fault tolerance and recovery behavior. The system should handle database connection failures gracefully, recover automatically when services restart, and maintain stability during extended operation. Memory leaks, resource exhaustion, and performance degradation over time are critical defects for a monitoring system that runs continuously.

The table below defines the performance and reliability test cases for PRISM.

ID	Test Name	Procedure	Pass Threshold	Priority	Notes
PRT-01	Agent CPU Overhead	Run collector for 10 minutes while monitoring with top/htop	Agent CPU usage < 1% average	High	Critical for production viability
PRT-02	Agent Memory Footprint	Monitor Python process RSS during extended collection	Memory usage < 50 MB steady state	High	Prevents resource exhaustion
PRT-03	API Response Time - Latest	Call /api/metrics/latest 100 times and measure latency	95th percentile < 200ms	High	Dashboard responsivene ss
PRT-04	API Response Time - History	Call /api/metrics/history?range=24h with 17,280 rows	Response time < 1 second	High	Large dataset handling
PRT-05	Dashboard Render Performance	Load dashboard with 24 hours of history data	Initial render < 2 seconds	Medium	User experience
PRT-06	Concurrent API Requests	Simulate 10 concurrent dashboard clients polling every 5 seconds	All requests complete successfully, no timeouts	Medium	Multi-user support
PRT-07	Database Write Throughput	Run collector at 1-second intervals for 5 minutes	All samples written without queue backup	High	Stress test collection

PRT-08	Extended Runtime Stability	Run full system continuously for 24 hours	No memory leaks, crashes, or degradation	Critical	Production reliability
PRT-09	Database Connection Recovery	Kill MySQL connection mid-operation, observe recovery	Collector reconnects and resumes within 30 seconds	High	Fault tolerance
PRT-10	Service Restart Recovery	Restart prism-document-agent via systemctl	Service resumes collection within 10 seconds	High	Operational continuity
PRT-11	Large Process List Handling	Test with 500+ active processes on system	Top processes endpoint responds < 500ms	Medium	Edge case performance
PRT-12	History Data Pruning	Accumulate 7 days of data, verify cleanup job	Old data removed, query performance maintained	Medium	Long-term stability

5.2.2 Security and Access Testing

Security and access testing verifies that PRISM protects sensitive operational data and enforces proper authentication and authorization. As a system monitoring tool, PRISM has visibility into process lists, system resource usage, and potentially sensitive operational patterns. This information must be protected from unauthorized access.

Security testing focuses on input validation to prevent injection attacks, proper authentication to ensure only authorized users can access the dashboard, role-based access control to restrict administrative functions, and protection of configuration secrets. Because PRISM stores data in MySQL and serves a web interface, it faces common web application security risks that must be mitigated.

The table below defines the security and access test cases for PRISM.

ID	Test Name	Attack Vector / Input	Expected Defense	Priority	Notes
SAT-01	SQL Injection - API Parameters	Send malicious SQL in range parameter: ?range=1h'; DROP TABLE cpu;--	Input rejected or sanitized, no database impact	Critical	Prepared statements required
SAT-02	SQL Injection - Process Limit	Send malicious input: ?limit=10; DELETE FROM process_info	Input rejected, returns error or default	Critical	Parameter validation

SAT-03	Authentication - Valid Credentials	Login with correct username and password	User authenticated, session/token issued	High	Basic auth flow
SAT-04	Authentication - Invalid Password	Login with correct username, wrong password	Access denied, error message shown	High	Reject bad credentials
SAT-05	Authentication - Nonexistent User	Login with username not in database	Access denied, no user enumeration hint	High	Prevent user discovery
SAT-06	Session Expiration	Use session token after configured timeout period	Request rejected, user redirected to login	Medium	Session security
SAT-07	Role-Based Access - Admin Features	Access admin controls as non-admin user	Admin features hidden or access denied	High	Authorization enforcement
SAT-08	API Without Authentication	Call /api/metrics/latest without auth token	Request rejected with 401 Unauthorized	High	Endpoint protection
SAT-09	XSS in Process Commands	Insert script tag in process command, view in dashboard	Script tags escaped, not executed	High	Output encoding
SAT-10	Configuration File Exposure	Request .env or config files via API or web paths	Files not accessible, 404 returned	Critical	Secrets protection
SAT-11	Database Credential Security	Check if DB password visible in logs or process list	Credentials not exposed in plain text	Critical	Credential management
SAT-12	CORS Policy Enforcement	Make API request from unauthorized origin	Request blocked by CORS policy	Medium	Cross-origin protection

5.2.3 Build and Deployment Testing

Build and deployment testing verifies that PRISM can be packaged, distributed, and installed correctly on target Ubuntu systems. The project includes Debian packaging scripts that create .deb files for distribution. These packages must include all required components, register systemd services correctly, and handle installation, upgrade, and removal scenarios properly.

Deployment testing is particularly important for PRISM because the system involves multiple components that must be installed and configured together: the Python collector agent, the Node.js API

server, the React dashboard, and the MySQL database schema. Failures in any of these areas would prevent successful deployment, even if all functional tests pass.

The table below defines the build and deployment test cases for PRISM.

ID	Test Name	Verification Method	Pass Criteria	Priority	Notes
BDT-01	Build Script Execution	Run scripts/build-full-deb.sh with default arguments	Script completes with exit code 0	Critical	Basic build verification
BDT-02	Build with Version Argument	Run build script with explicit version: ./build-full-deb.sh 1.2.3	Package created with correct version in control file	High	Version management
BDT-03	Debian Control File Generation	Inspect DEBIAN/control in built package	Contains correct package name, version, dependencies	High	Package metadata
BDT-04	Package File Structure	Extract .deb and verify contents	All runtime files present in expected locations	High	Complete packaging
BDT-05	Clean Install on Ubuntu	Install .deb on fresh Ubuntu 20.04+ system with apt	Package installs without dependency errors	Critical	Target platform support
BDT-06	postinst Script Execution	Monitor postinst during installation	Database initialized, services registered	High	Post-install automation
BDT-07	prerm Script Execution	Uninstall package and verify prerm runs	Services stopped, cleanup performed	High	Clean uninstallation
BDT-08	systemd Service Registration	Check systemctl status after install	prism-document-agent and prism-document-web registered	Critical	Service management
BDT-09	Service Auto-Start on Boot	Reboot system after installation	Both services start automatically	High	Production readiness
BDT-10	Configuration File Deployment	Verify config files in /etc/prism/ after install	Environment files present with correct permissions	High	Configuration management

BDT-11	Schema SQL Deployment	Check for schema.sql in package	Schema file present and valid SQL syntax	Medium	Database setup support
BDT-12	Upgrade Installation	Install new version over existing installation	Upgrade succeeds, data preserved, services restart	High	Version upgrades
BDT-13	Dashboard Static Assets	Verify React build files in package	All JS, CSS, and assets present in web root	High	Frontend deployment
BDT-14	Python Dependencies	Check virtual environment and requirements	All Python packages installed in venv	High	Agent dependencies
BDT-15	Node.js Dependencies	Verify node_modules in server directory	All npm packages present for API server	High	API dependencies

6. Test Data

Testing will use a combination of live system metrics, controlled input conditions, and negative test cases. Representative scenarios will include normal system activity, elevated CPU load, low available memory, increased disk usage, and process lists with multiple active entries. Invalid credentials, unavailable database connections, missing environment files, and failed service starts will also be used to verify error handling.

7. Entry and Exit Criteria

Entry Criteria

- Code is available in the target branch or release candidate.
- Required environment dependencies are installed.
- Database schema and configuration files are available.
- Build script and service files are present for deployment tests.

Exit Criteria

- Critical and high-severity functional defects are resolved or formally accepted.
- End-to-end data flow works correctly in the target environment.
- Build, installation, and service startup complete successfully.
- Test results are documented and reviewed.

8. Risks Relevant to Testing

Risk	Potential Test Impact	Test Response
Database unavailable	Collected metrics may fail to persist; end-to-end tests may break	Run failure-path tests and verify logging, restart, and recovery handling
Agent overhead affects host	Monitored results may be skewed by the monitor itself	Measure agent resource use during runtime and compare against target thresholds
Large data volume over time	History queries and dashboard views may slow down	Run tests with larger datasets and validate retention or pruning behavior
Configuration errors after install	Services may not start or may connect with invalid credentials	Include post-install configuration and negative startup scenarios

9. Test Deliverables

- Functional and integration test cases
- Test execution results and logs
- Defect list and resolution tracking
- Build and installation verification evidence
- Final test summary report

10. Conclusion

This test approach is intended to verify that PRISM works as a complete, deployable monitoring system rather than as a collection of unrelated parts. The most important success criteria are accurate metric collection, correct data flow across all layers, reliable dashboard behavior, and successful packaging and installation on Ubuntu. A structured mix of function testing, integration testing, system testing, and deployment validation will provide confidence that the product is ready for demonstration and delivery.